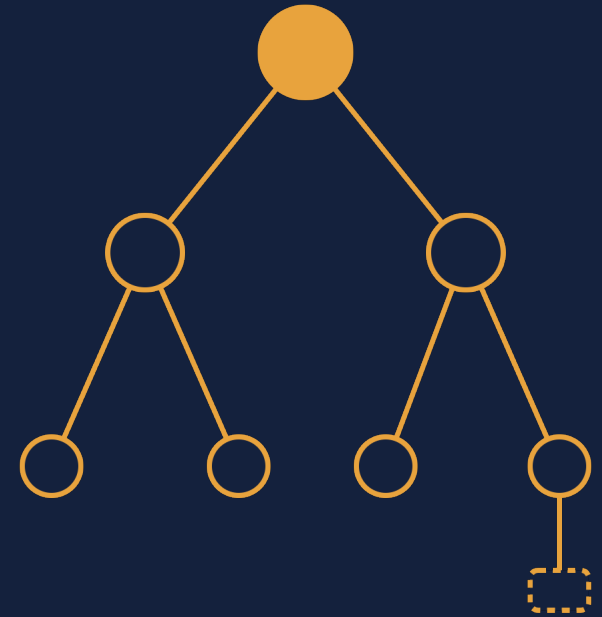


DWEC · UNIDAD 3.1

Páginas web dinámicas

El DOM y la selección de elementos con JavaScript



ÍNDICE

Qué vamos a ver

01

Contenido
dinámico

02

El DOM y sus
nodos

03

Métodos de
selección

04

Selectores CSS

05

Navegar por el
árbol

Objetivo: saber **localizar cualquier elemento** de una página para, en la siguiente unidad (3.2), poder modificarlo.

¿Dónde se genera el contenido?

Backend (servidor)

El servidor construye el HTML (PHP, Node, Java...) y lo envía ya terminado al navegador.

ESTA UNIDAD

Frontend (navegador)

JavaScript modifica la página **después** de cargarla: añade, cambia o elimina contenido sin recargar.

La pieza clave para hacerlo es el **DOM**: una representación en árbol de la página que JavaScript puede leer y modificar.

Document Object Model

Cada vez que abres una página, el navegador crea **en memoria** un modelo de objetos que la representa: el **árbol DOM**.

Jerárquico

Refleja el anidamiento del HTML: cada etiqueta dentro de otra es una rama hija.

Hecho de nodos

Cada pieza del árbol (documento, etiqueta, atributo, texto) es un nodo.

Raíz arriba

El árbol empieza en el nodo documento y «crece» hacia abajo hasta las hojas.

Los 4 tipos de nodo que más usarás

`document`

Documento

Toda la página. Es la raíz del árbol y el punto de entrada: el objeto global `document`.

`<table>`

Elemento

Cada etiqueta HTML: `<main>`, `<h1>`, `<table>`, `<tbody>`...

`id="main"`

Atributo

Los atributos de las etiquetas: `lang`, `id`, `summary`, `class`...

`"Contactos"`

Texto

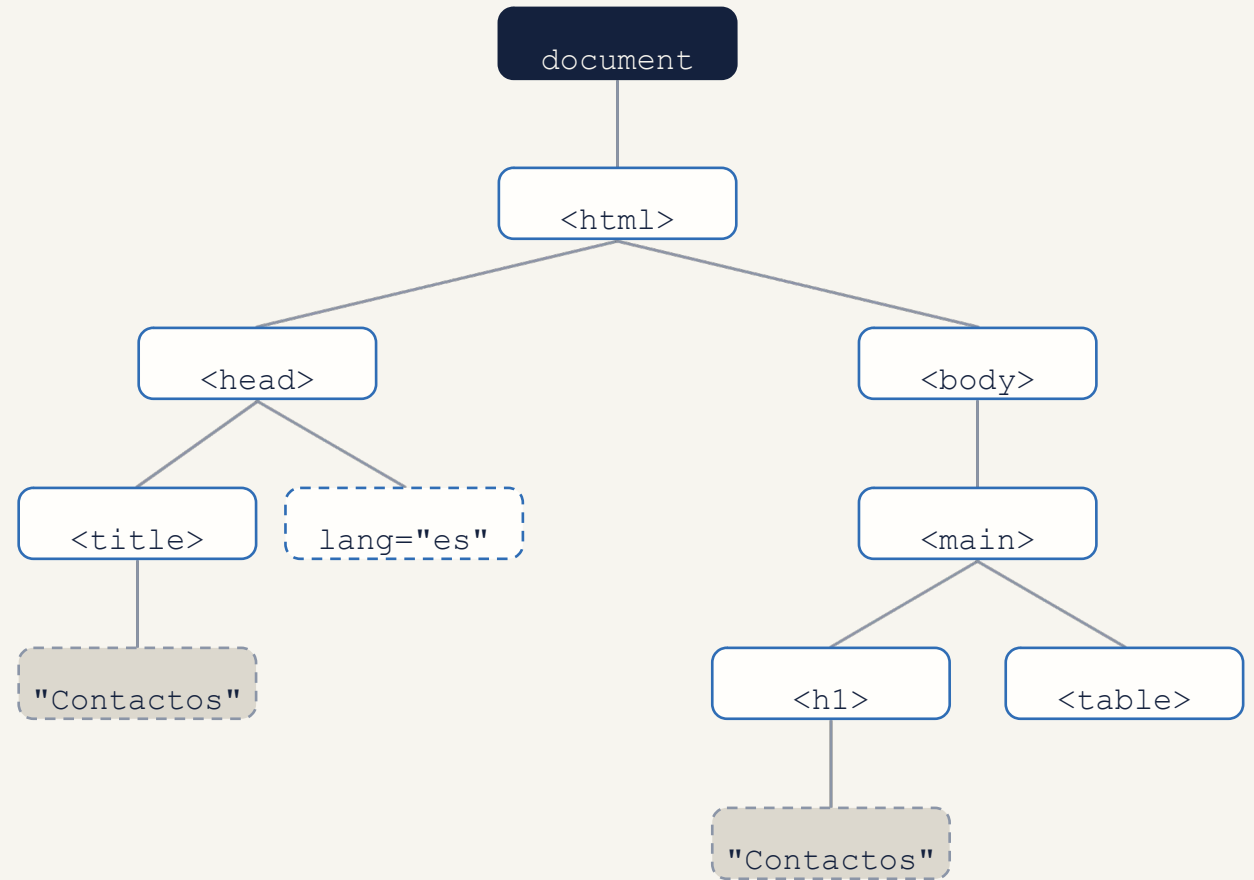
El texto dentro de un elemento. No tiene hijos: siempre es una hoja.

Existen 12 tipos de nodo en total, pero estos cuatro cubren casi todo el trabajo diario.

2 · EL DOM

Del HTML al árbol DOM

```
<!DOCTYPE html>
<html>
  <head lang="es">
    <title>Contactos</title>
  </head>
  <body>
    <main id="main">
      <h1>Contactos</h1>
      <table id="tabla-contactos">
        <!-- thead, tbody... -->
      </table>
    </main>
  </body>
</html>
```



PRUÉBALO EN CLASE

Inspecciona el DOM en el navegador

1

Abre las DevTools

F12 o Ctrl + Mayús + I en Chrome.

2

Pestaña «Elementos»

Muestra el DOM vivo, no el archivo HTML.

3

Edita un nodo

Doble clic sobre un texto o atributo para cambiarlo.

4

Recarga la página

Los cambios desaparecen: el archivo no se modifica.

Muy útil para probar ideas rápidamente... pero lo que cambies aquí no se guarda.

El nodo documento: *document*

Propiedad	Contiene
<code>document.title</code>	Título del documento
<code>document.lastModified</code>	Fecha de última modificación
<code>document.URL</code>	URL del documento
<code>document.domain</code>	Dominio del documento
<code>document.cookie</code>	Cookies del documento
<code>document.forms</code>	Todos los formularios
<code>document.images</code>	Todas las imágenes
<code>document.links</code>	Todos los enlaces

```
console.log(document.title);  
// 'Contactos'  
  
console.log(document.images.length);  
// n° de imágenes
```

Es el **punto de entrada** para cualquier trabajo con el DOM.

La API DOM

El W3C especifica qué propiedades, métodos y tipos de nodo existen. Es independiente del lenguaje: hay implementaciones en JavaScript, Java, C++...

Interfaz

Define qué métodos deben existir (el «contrato»).

Implementación

La realización concreta de una interfaz; aquí, el navegador.

API

Conjunto de interfaces que ofrece un sistema. La API DOM es la que usamos para trabajar con páginas.



En Node.js no existe *document*. El DOM solo está en el navegador; en Node hay que usar librerías como **jsdom**.

3 · SELECCIÓN

Mapa de los métodos de selección

Antes de cambiar o añadir algo, primero hay que **seleccionar** el elemento.

Método	Devuelve	Se invoca sobre
<code>getElementById()</code>	Un elemento o null	Solo document
<code>getElementsByClassName()</code>	HTMLCollection viva	document y elementos
<code>getElementsByTagName()</code>	HTMLCollection viva	document y elementos
<code>getElementsByName()</code>	NodeList viva	Solo document
<code>querySelector()</code>	Un elemento o null	document y elementos
<code>querySelectorAll()</code>	NodeList estática	document y elementos

Viva = se actualiza sola si el DOM cambia · Estática = foto fija del momento de la consulta.

3 · SELECCIÓN

Por ID: getElementById()

```
const elementoPrincipal = document.getElementById('main');  
elementoPrincipal.className = 'borde'; // le asigna la clase CSS 'borde'
```

Sin almohadilla

Se pasa el id como cadena: 'main', no '#main'.

Uno o ninguno

Devuelve un único elemento... o **null** si no existe ese id.

El más rápido

Los id son únicos, así que el navegador lo encuentra enseguida.

3 · SELECCIÓN

Programación defensiva: ¿y si es null?

1 · if clásico

Lo más legible

```
if (elementoPrincipal !== null) {  
  elementoPrincipal.className = 'borde';  
}
```

2 · Operador &&

Solo evalúa la derecha si la izquierda es verdadera

```
elementoPrincipal && (elementoPrincipal.className = 'borde');
```

3 · Encadenamiento ?.

ES2020 · solo para leer o llamar

```
elementoPrincipal?.classList.add('borde');    // válido  
// elementoPrincipal?.className = 'borde';    // SyntaxError
```

3 · SELECCIÓN

Guarda la selección en una variable

Evitar: dos búsquedas en el árbol

```
document.getElementById('main').className = 'borde';  
document.getElementById('main').style.color = 'navy';
```

Mejor: una búsqueda, reutilizada

```
const principal = document.getElementById('main');  
principal.className = 'borde';  
principal.style.color = 'navy';
```

Cada llamada a un método DOM cuesta tiempo de cálculo. Por clase o etiqueta es más lento que por id: hay que recorrer todo el árbol.

3 · SELECCIÓN

Por clase: `getElementsByClassName()`

```
const filasPares = document.getElementsByClassName('par');  
console.log(filasPares.length);    // 2  
console.log(filasPares[0]);        // <tr class="fila par">...</tr>
```

Parece un array...

Tiene índice `[i]`, `.length` y `.item(i)`. Mantiene el orden del documento.

...pero no lo es

Es un **HTMLCollection**: no tiene `map`, `filter` ni `forEach`. Confundirlo es un error típico.

3 · SELECCIÓN

Cuatro formas de recorrer una colección

1 · for con índice

```
for (let i = 0; i < filas.length; i++) {  
  filas[i].style.backgroundColor = '#CCC';  
}
```

2 · for...of (ES6)

```
for (const fila of filas) {  
  fila.style.backgroundColor = '#CCC';  
}
```

3 · Préstamo de métodos

```
Array.prototype.forEach.call(filas, (fila) => {  
  fila.style.backgroundColor = '#CCC';  
});
```

4 · Convertir en array (ES6)

```
Array.from(filas).forEach((fila) => {  
  fila.style.backgroundColor = '#CCC';  
});
```

3 · SELECCIÓN

Cambiar el CSS con la propiedad style

Cada propiedad CSS es una propiedad del objeto **style**. Las que llevan guion pasan a **camelCase**.

CSS	JavaScript
background-color	style.backgroundColor
font-family	style.fontFamily
font-size	style.fontSize
border	style.border

```
const titulo = document.querySelector('h1');  
titulo.style.color = '#2F6DB5';  
titulo.style.fontSize = '48px';
```

style escribe estilos en línea. Para cambios de aspecto completos, suele ser mejor cambiar la clase.

3 · SELECCIÓN

Colecciones vivas vs. estáticas

Viva

`getElementsByClassName()`, `getElementsByTagName()`, `getElementsByName()`. Si el DOM cambia, la colección se actualiza sola.

Estática

`querySelectorAll()`. Es una foto fija: no refleja los elementos que se añadan o quiten después.

```
const vivas = document.getElementsByTagName('li');
const estaticas = document.querySelectorAll('li');
console.log(vivas.length, estaticas.length);    // 8 8

document.querySelector('ul').append(document.createElement('li'));    // (UT 3.2)
console.log(vivas.length, estaticas.length);    // 9 8
```

3 · SELECCIÓN

Por etiqueta: `getElementsByTagName()`

```
const celdas = document.getElementsByTagName('td');  
for (const celda of celdas) {  
  celda.style.fontFamily = 'Verdana';  
  celda.style.fontSize = '9pt';  
}
```



Pasa solo el nombre: **'td'**. Con **'<td>'** no encuentra nada. Además es poco específico: suele traer muchos elementos.

3 · SELECCIÓN

Por atributo name: `getElementsByName()`

```
<fieldset>
  <input type="radio" id="st" name="genero" value="Stonerrock">
  <input type="radio" id="sp" name="genero" value="Spacerock">
  <input type="radio" id="ha" name="genero" value="Hardrock">
</fieldset>
```

```
const opcionesGenero = document.getElementsByName('genero');
for (const opcion of opcionesGenero) {
  console.log(opcion.value); // Stonerrock, Spacerock...
}
```

Úsalo con cuidado

Algunos navegadores antiguos también devolvían elementos por id. Alternativa: `querySelectorAll('[name="genero"]')`.

4 · SELECTORES CSS

querySelector() y querySelectorAll()

```
// Primer elemento que encaje con el selector (o null)
const celda = document.querySelector('#main table td:nth-child(2)');
celda.style.border = 'thick solid red';

// Todos los que encajen: NodeList estática, que SÍ tiene forEach
const celdas = document.querySelectorAll('#main table td:nth-child(2)');
celdas.forEach((celda) => celda.style.border = 'thick solid red');
```

querySelector()

Solo la segunda celda de la **primera** fila.

querySelectorAll()

La segunda celda de **todas** las filas: la columna de apellidos.

¿Por qué molan?

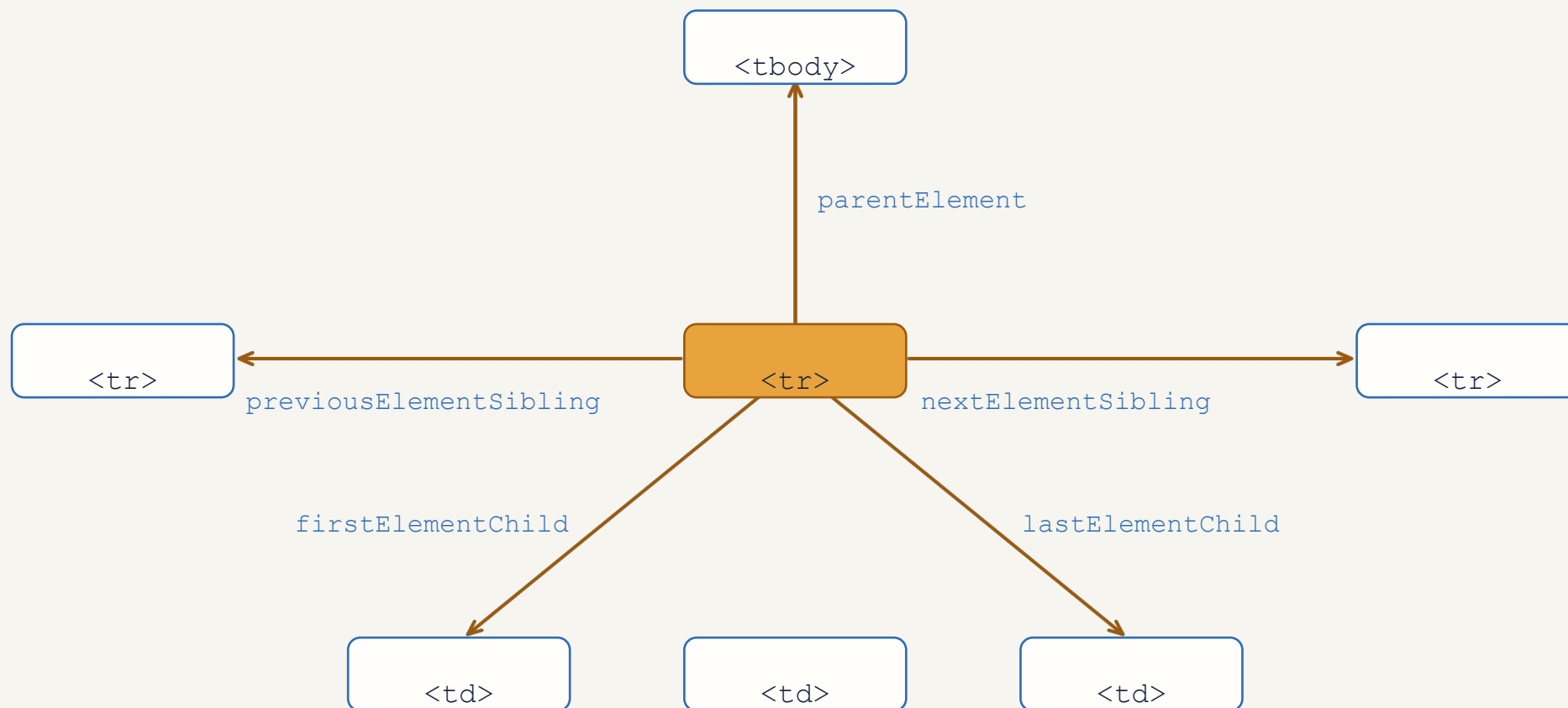
Aceptan cualquier selector CSS: los más versátiles y expresivos.

Chuleta de selectores útiles

Selector	Selecciona
*	Todos los elementos
E	Elementos de tipo E
.clase	Con esa clase
#id	Con ese id
[a]	Con el atributo a
[a="b"]	Atributo a igual a b
[a^="b"]	Atributo a empieza por b
[a*="b"]	Atributo a contiene b

Selector	Selecciona
E F	F descendiente de E
E > F	F hijo directo de E
E + F	F justo después de E
E ~ F	F hermano posterior de E
:nth-child(n)	Hijo n-ésimo
:first-child	Primer hijo
:not(s)	Que NO cumpla s
:checked	Radio o casilla marcada

Moverse desde un elemento conocido



¿Nodo o elemento?

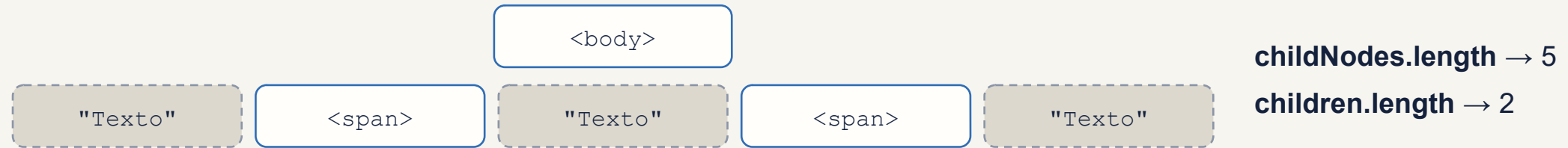
Cualquier nodo	Solo elementos
parentNode	parentElement
childNodes	children
firstChild	firstElementChild
lastChild	lastElementChild
previousSibling	previousElementSibling
nextSibling	nextElementSibling

Todo elemento es un nodo, pero no todo nodo es un elemento.

Casi siempre querrás la columna de la derecha.
Excepción curiosa: el padre de <html> es el nodo documento, así que su **parentElement** es **null**.

5 · NAVEGAR POR EL ÁRBOL

Cuidado con los nodos de espacio en blanco



```
const cuerpoTabla = document.querySelector('tbody');
console.log(cuerpoTabla.children.length); // 4 (las filas <tr>)
console.log(cuerpoTabla.childNodes.length); // 9 (4 <tr> + 5 textos vacíos)
console.log(cuerpoTabla.firstChild); // #text (salto de línea)
console.log(cuerpoTabla.firstElementChild); // <tr>
```

Los saltos de línea y tabulaciones del HTML generan nodos de texto «vacíos».

5 · NAVEGAR POR EL ÁRBOL

Buscar solo dentro de un subárbol

```
const todas = document.getElementsByTagName('li');  
console.log(todas.length);           // 8  
  
const subLista = document.getElementById('lista-2');  
const entradas = subLista.getElementsByTagName('li');  
console.log(entradas.length);        // 4
```

Llamado sobre un elemento, el método busca solo **por debajo** de él. Excepción: `getElementById()` y `getElementsByName()` solo existen en `document`.

- Entrada 1

- Entrada 2

- ul#lista-2

- Entrada 2.1

- Entrada 2.2

- Entrada 2.3

- Entrada 2.4

- Entrada 3

- Entrada 4

Atajos de document por tipo

Propiedad	Da acceso a
<code>document.anchors</code>	Todos los anclajes
<code>document.forms</code>	Todos los formularios
<code>document.images</code>	Todas las imágenes
<code>document.links</code>	Todos los enlaces
<code>document.head</code>	El elemento <head>
<code>document.body</code>	El elemento <body>

```
console.log(document.forms.length);  
document.body.style.backgroundColor =  
  '#F7F5EF';
```

RESUMEN

¿Qué uso en cada caso?

Necesito...	Uso
Tengo el id	<code>getElementById('id')</code>
Un elemento con cualquier selector CSS	<code>querySelector('sel')</code>
Varios elementos para recorrer con <code>forEach</code>	<code>querySelectorAll('sel')</code>
Moverme desde un elemento que ya tengo	<code>parentElement</code> , <code>children</code> , <code>nextElementSibling...</code>
Una colección que se actualice sola	<code>getElementsByClassName</code> / <code>TagName</code>

Y siempre: **comprueba null** y **guarda la selección** en una variable.